Machine Learning Course Project-I

Industrial Machine Condition Classification

Team Name: Carrot

Name	Roll number	Contribution
Aradhya Chakrabarti	MT2026012	• Data loading, cleaning and exploratory analysis for both competitions • Feature engineering (route, airline code, flight number, stops; temperature difference and load ratios) • Fold-safe preprocessing, training and cross-validation of all model families • Hyper-parameter tuning and tree pruning • Diagnostics (residuals, importances, confusion matrices, ROC, t-SNE) • Kaggle submissions and report drafting
Ankit Lamba	MT2026XXX	`<contribution>`
Kunal Yadav	MT2026XXX	`<contribution>`
Kshitij Beri	MT2026XXX	`<contribution>`

Source code

The full code for this competition is in the project repository: `https://github.com/TimeATronics/AIT511_Project1`

1 Problem and dataset

We are given a classification problem: classify a machine into one of three states using sensor measurements, with 0 for normal, 1 for thermal failure and 2 for mechanical failure. Thermal failures are driven by temperature and cooling conditions, and mechanical failures by load, wear, torque and vibration.

The training file `student_dataset.csv` has 16,000 rows, and the test file has 4,000 rows. The columns are unit_type (S/M/H) and ten sensors: ambient_temp, process_temp, coolant_flow, shaft_speed, drive_torque, power_load, component_wear, thermal_stress, mechanical_load and vibration_index. The column unit_id identifies the machine (16,000 unique values, none of them in the test set) and is never used as a feature. The evaluation metric is accuracy.

We have used the following models: logistic regression (linear and quadratic basis), LDA, QDA, Gaussian naive Bayes, decision trees, and random forests.

2 Data loading

We load the three CSV files (training, test and the sample submission) with pandas, print their shapes, and build a small overview frame with the row count, column count, duplicate rows and missing cells. There are no duplicate rows, but every column has a small number of missing sensor readings, so we handle them during preprocessing and remove no rows.

3 Exploratory data analysis

3.1 Class distribution

For EDA, the first thing we check is the class balance, since an uneven split can make accuracy look better than the model actually is. A count block and a bar plot show the classes are 8,000 / 4,000 / 4,000, that is a 2:1:1 split. A model that always predicts "normal" would already score 50% accuracy, so we report balanced accuracy and per-class metrics alongside accuracy.

3.2 Sensor analysis by class

Next we look at how the sensors behave for each class, since the failures are supposed to follow the physics. The code computes the per-class sensor means and draws the mean bars and boxplots (Figures 1 and 2). Accordingly, we observe the following:

- Thermal failures (class 1) are hotter: mean process_temp 72.4 against 54.7 for normal and 62.2 for mechanical, and mean thermal_stress 37.9 against 21.3 for normal.
- Mechanical failures (class 2) show more wear and load: mean component_wear 165.0 against 87.4 for normal, mechanical_load 88.0 against 64.8, and vibration_index 2.10 against 1.51.
- Normal machines (class 0) have the lowest values of these variables, and the boxes for the two failure classes overlap the least exactly where the physical readings differ the most.

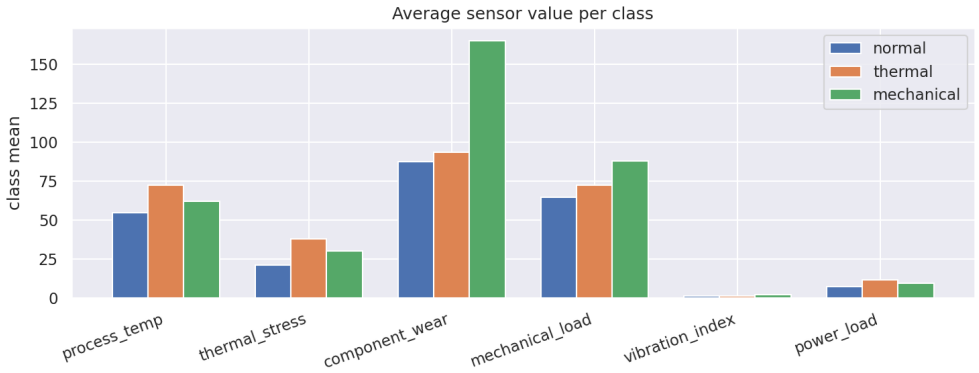


Figure 1: Mean sensor readings per class for six key sensors.

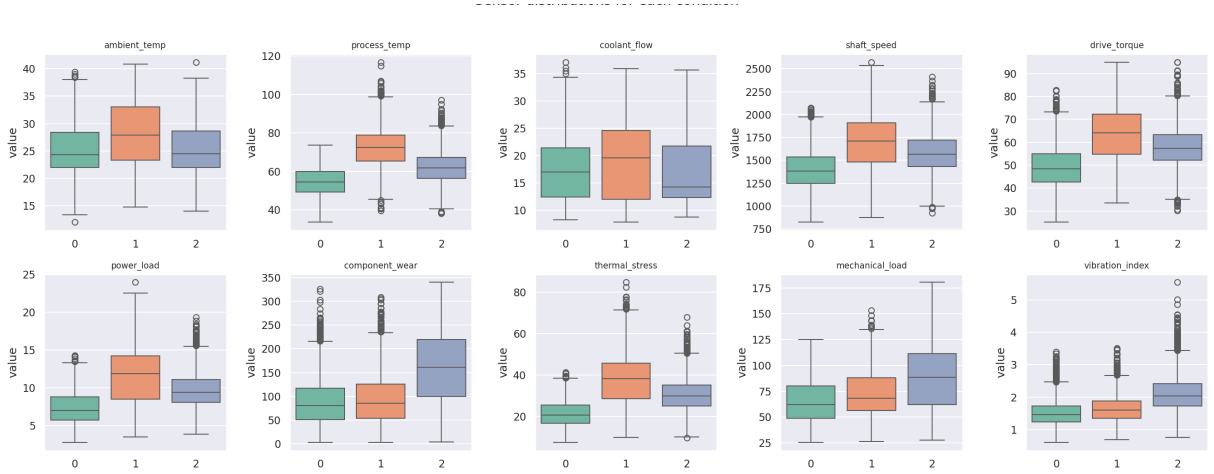


Figure 2: Every sensor by class.

3.3 Missing value analysis

We then check how many values are missing in each column, in order to decide how to fill them. Missing values are few everywhere: the worst is vibration_index, which loses 0.46% of its values (74 rows), and the others fall between 0.14% and 0.38%. With numbers this small, filling the gaps with the column median is safe and no rows need to be dropped. unit_type has 22 missing entries (0.14%), which are filled with the most common type.

3.4 Correlation analysis

Next we check the correlations between the sensors, because they decide whether models that assume independent features are suitable. The heatmap block prints the strongest pairs as well. Fifteen pairs correlate above 0.5: the strongest are power_load with thermal_stress (0.93), drive_torque with power_load (0.92), and process_temp with thermal_stress (0.86). The sensors are therefore far from independent, which matters for the naive Bayes classifier later.

3.5 Machine type analysis

Finally we check the machine type, since failure rates may depend on it. The type is informative: 42.4% of H machines are thermal failures, while S machines lean towards mechanical failures (29.4%), so we keep the type as a one-hot feature.

4 Data cleaning

There are no impossible values and no duplicate rows, so no rows are removed. The missing sensor readings are handled by the imputer, and unit_id is set aside to line up the submissions.

5 Feature engineering

Since the EDA showed that the two failure classes differ mainly in temperature and in wear and load, we add two kinds of extra features with one function applied to train and test:

1. `temp_difference` = process_temp − ambient_temp. Thermal failures are driven by the machine being hotter than its surroundings, and the difference expresses exactly that. It is a linear combination of two existing sensors, which is fine for trees and logistic regression but matters for QDA (Section 9).

2. `power_per_speed` = power_load / shaft_speed, and `torque_per_wear` = drive_torque / component_wear: rough stress-per-revolution and stress-per-wear ratios that match the mechanical failure readings. Division by zero is left as a missing value and handled by the imputer.

6 Data preprocessing

All preprocessing is built with a scikit-learn `ColumnTransformer` and used inside a `Pipeline`, so during cross-validation the transformations are fitted on the training folds only.

- Missing values: the median for numeric columns, the most frequent value for unit_type.
- Encoding: unit_type is one-hot encoded, with `handle_unknown="ignore"` in case the test set contains a type that training did not see.
- Scaling: standardisation for the models that need it (logistic regression, LDA, QDA, naive Bayes); the tree and forest use the raw values.

7 Model evaluation

We report accuracy, balanced accuracy and macro F_1. For a two-class problem with predictions $\hat{y}$ and labels y,

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \qquad \text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \qquad F_1 = \frac{2\,\text{precision}\,\text{recall}}{\text{precision} + \text{recall}},$$

and the multi-class versions average these per-class values (macro F_1). Balanced accuracy is the average of the per-class recalls, so it treats the three classes as equally important even when the split is 2:1:1. The ROC curve plots

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad \text{against} \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}},$$

and the area under it (AUC) summarises the ranking quality across all thresholds. All metrics come from code blocks that call `sklearn.metrics` on out-of-fold predictions.

For validation we split the training data into $K = 5$ stratified folds, so every fold keeps the class proportions, and average the five scores: $\frac{1}{K}\sum_{k=1}^{K} e_k$. The finalists are re-checked with 5 folds repeated 3 times (15 scores each). A fixed random state (42) is used for every fold split and every model, so the results repeat.

8 Model family direct comparisons (baseline)

We first train each classifier from the notes once with reasonable settings:

- **Bayes rule**: the posterior follows from the prior and the likelihood,

  $$p(y \mid x) = \frac{p(x \mid y)\, p(y)}{p(x)}.$$

  The Bayes optimal classifier predicts the most probable class,

  $$f^*(x) = \arg\max_y \, p(y)\, p(x \mid y).$$

- **LDA**: Gaussian class-conditional densities with a shared covariance matrix, giving linear boundaries.
- **QDA**: Gaussian class-conditional densities with a covariance per class, giving quadratic boundaries.
- **Gaussian naive Bayes**: independent Gaussian features per class, $p(x \mid y) = \prod_{j=1}^{d} p(x_j \mid y)$.
- **Logistic regression**: models the posterior directly, $\eta(x) = \sigma(w^\top x) = 1/(1+e^{-w^\top x})$, fitted by maximum likelihood.
- **Decision tree**: recursive binary splitting with the Gini impurity, $\text{Gini}(S) = 1 - \sum_k p_k^2$.
- **Random forest**: an average vote over many decorrelated trees.

The code blocks build these six pipelines, evaluate them with stratified 5-fold cross-validation and draw the comparison chart (Figure 3).

Model	CV accuracy
Random forest (300)	0.8776
QDA (raw sensors)	0.8476
Logistic regression (linear)	0.8280
LDA	0.8239
Decision tree (unrestricted)	0.8056
Gaussian naive Bayes	0.7472

The forest leads at 0.878, and QDA is the strongest of the three Gaussian models at 0.848, which fits the EDA: the failure regions are roughly elliptical, so letting each class have its own covariance pays off. Plain logistic regression (0.828) and LDA (0.824) are below QDA because their boundaries are linear, and Gaussian naive Bayes is last at 0.747 because it assumes independent sensors while several pairs are correlated.

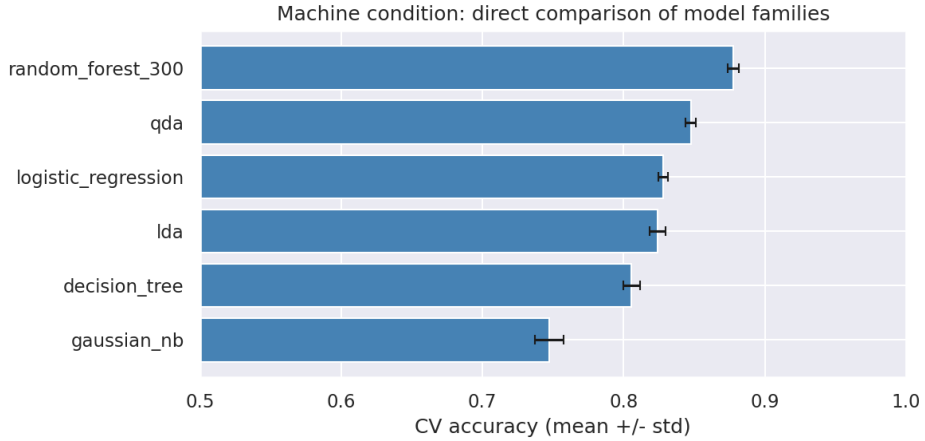


Figure 3: Direct comparison of the model families (CV accuracy).

9 Hyper-parameter tuning

9.1 Logistic regression (linear): regularisation strength

We start with the linear logistic model,

$$\eta(x) = P(Y = 1 \mid X = x) = \sigma(w^\top x) = \frac{1}{1 + e^{-w^\top x}}, \qquad \ln \frac{\eta}{1 - \eta} = w^\top x,$$

fitted by minimising the negative log-likelihood $\mathcal{L}(w) = \sum_n \ln\!\left(1 + e^{-y_n w^\top x_n}\right)$, whose gradient is $\partial\mathcal{L}/\partial w = \sum_n (\sigma_n - y_n) x_n$. There is no closed form, so the solver iterates. The strength is set through `C`, the inverse of the L2 penalty, and we sweep it. The results are: $C = 0.01$ gives 0.818, $C = 0.1$ gives 0.825, $C = 1$ gives 0.828, $C = 10$ gives 0.829 and $C = 100$ gives 0.829. Accuracy rises up to $C = 10$ and stays flat, so the data supports a weak penalty.

9.2 Logistic regression with a quadratic basis

Since the boundaries in this dataset are not linear, we give the linear model squared and interaction terms of the sensors. This lets the linear decision function bend into a quadratic boundary, the same hypothesis family as QDA but fitted discriminatively; the code builds a pipeline with `PolynomialFeatures(degree=2)` before the logistic model. We try degree 1 versus 2 and $C \in \{0.01, 0.1, 1, 10\}$. Degree 2 with $C = 1$ reaches 0.8853, clearly above the linear version (0.829), and the degree-1 rows repeat the previous sweep as a control.

9.3 QDA: covariance regularisation

QDA estimates one covariance matrix per class:

$$p(x \mid y) = \frac{1}{(2\pi)^{d/2}|\Sigma_y|^{1/2}} \exp\Big(-\tfrac{1}{2}(x-\mu_y)^\top \Sigma_y^{-1}(x-\mu_y)\Big),$$

and the Bayes rule classifies by $\arg\max_y \big[\ln p(y) - \tfrac{1}{2}\ln|\Sigma_y| - \tfrac{1}{2}(x-\mu_y)^\top\Sigma_y^{-1}(x-\mu_y)\big]$, a quadratic boundary. A small regularisation is added to each covariance for numerical stability. The code block first shows what happens without it: with `reg_param` $= 0$ the fit crashes because `temp_difference` is an exact linear combination of two sensors and the class covariance is singular. With regularisation the best QDA on the engineered features reaches only 0.8393, below the 0.8476 obtained from the raw sensors at `reg_param` $= 0.1$: the redundant column dilutes the covariance estimate even when the matrix stays invertible. We therefore use the raw-sensor version as the QDA representative in the final tables.

9.4 Gaussian naive Bayes: variance smoothing

The classifier assumes the sensors are conditionally independent given the class and adds a small smoothing value to every variance. We sweep the smoothing value from 10^{-9} to 0.1. The best setting (0.1) increases the score from 0.747 to 0.754. Naive Bayes stays the weakest family by a wide margin, since the independence assumption does not hold for these sensors.

9.5 Decision tree: pre-pruning and cost-complexity pruning

For pre-pruning we use a grid block over the same four growth limits as the regression tree (depth, leaf size, split size, maximum leaves). The best pre-pruned tree combines depth 12 with a minimum leaf size of 5 and scores 0.8389. The unrestricted tree falls to 0.806, and depth-4 trees underfit around 0.772. For post-pruning we use the cost-complexity criterion

$$R_\lambda(T) = R(T) + \lambda|T|,$$

with candidate λ values from the pruning path. Here it helps: the best λ (about 3×10^{-4}, keeping 161 leaves) lifts the tree from 0.8389 to 0.8429 (Figure 4), while larger λ values remove useful branches.

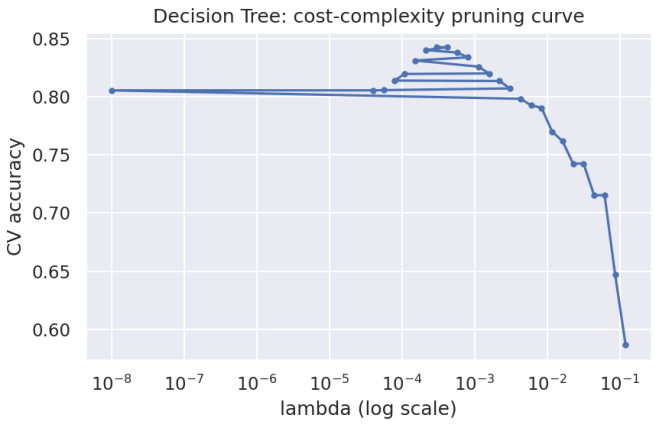


Figure 4: Cost-complexity pruning curve for the classification tree.

9.6 Random forest

The grid tunes the per-tree depth, the minimum leaf size and the class weighting. We include class weights because the classes are split 2:1:1. The unweighted depth-14 forest is the best at

0.8787, but the differences inside the unweighted block are tiny (0.876 to 0.879). Every `balanced` variant scores lower than the matching unweighted run, the first hint that reweighting trades overall correctness for minority-class attention.

10 Experiment: class imbalance handling

Since the 2:1:1 split could in principle hurt the two failure classes even while accuracy looks fine, we compare three standard treatments on the two leading models (the quadratic logistic regression and QDA): reweighting the classes, random oversampling of the minority classes, and SMOTE, which creates synthetic minority samples by interpolating between neighbours. The samplers sit inside imbalanced-learn pipelines after preprocessing, so they only ever see training folds. We record both accuracy and macro F_1, since accuracy alone could hide a poor minority class.

Table 1: Imbalance strategies for the quadratic logistic (5-fold CV).

Variant	Accuracy	Macro F_1
none	0.8853	0.8756
reweight (class weights)	0.8743	0.8666
random oversampling	0.8728	0.8649
SMOTE	0.8734	0.8660

Each of these experiment scenarios lowered both metrics. With classes that are already well separated and only a 2:1:1 split, the resampled training set biases the boundary towards synthetic or duplicated minority points without improving any real case, so we keep the unweighted models.

11 Model diagnostics

Confusion matrices. We compute out-of-fold predictions for the three leading models and draw their confusion matrices (Figure 5), with labels 0 normal, 1 thermal, 2 mechanical. With all the models, normal machines are recognised best (7,498 of 8,000 for the quadratic logistic). The two failure classes each lose between 660 and 670 machines, and most of those misses fall into the normal class, roughly 470 to 480 per class. Only about 180 to 200 machines swap between thermal and mechanical, and those two classes are the ones whose sensor boxes overlap the most.

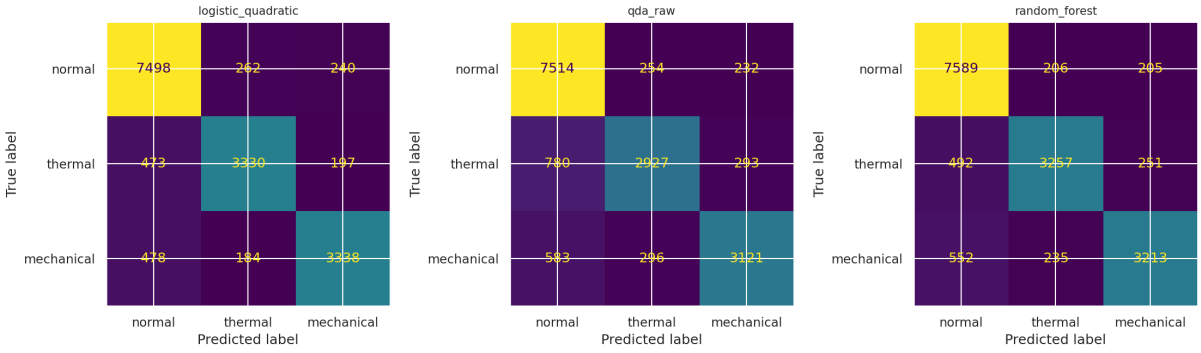
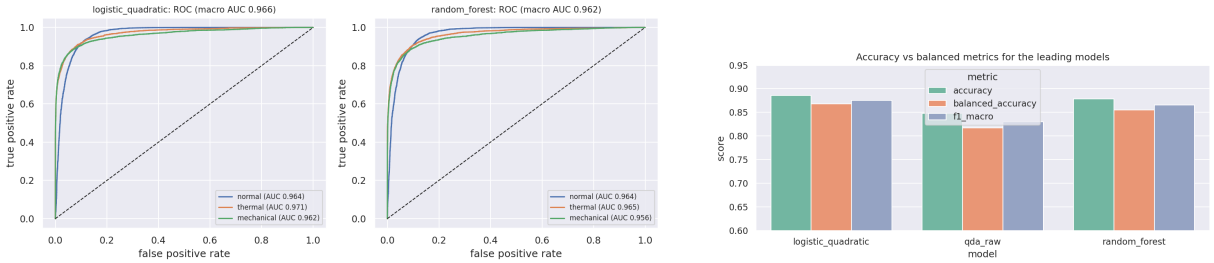


Figure 5: Out-of-fold confusion matrices for the three leading models.

Per-class metrics. We then print the full classification report for the quadratic logistic regression. Recall is 93.7% on normal, 83.3% on thermal and 83.5% on mechanical, with precision and F_1 following closely. No class collapses: the weakest recall is still above 80%.

ROC curves. Next we draw one-vs-rest ROC curves from the out-of-fold class probabilities and print the area under each curve. Both models have almost identical overall discrimination (macro AUC 0.966 for the quadratic logistic against 0.962 for the forest). The curves hug the top-left corner, and for the quadratic logistic the thermal class has the highest AUC at 0.971: even though thermal failures are the hardest to label correctly at the default threshold, they are the easiest to separate once the whole probability scale is used.

Balanced metrics. Finally we compute accuracy, balanced accuracy and macro F_1 for the three models and plot them side by side (Figure 6). The quadratic logistic regression is best on all three metrics (0.8854, 0.8681 and 0.8756), and the forest is second on all three. Since the ranking is the same under plain and balanced metrics, the headline accuracy does not come from the larger normal class alone.



(a) one-vs-rest ROC **(b)** accuracy vs balanced metrics

Figure 6: Discrimination and class-balance checks.

t-SNE. We also draw a t-SNE projection of a 3,000-row sample, before and after feature engineering (Figure 7), as an illustration of how the classes sit relative to each other. On the raw sensors the three classes form broad overlapping clouds, with mechanical machines mostly in the upper region and thermal machines largely on the right. After adding the engineered features the thermal cloud tightens on the right and the mechanical group separates more cleanly from the normal machines in the middle.

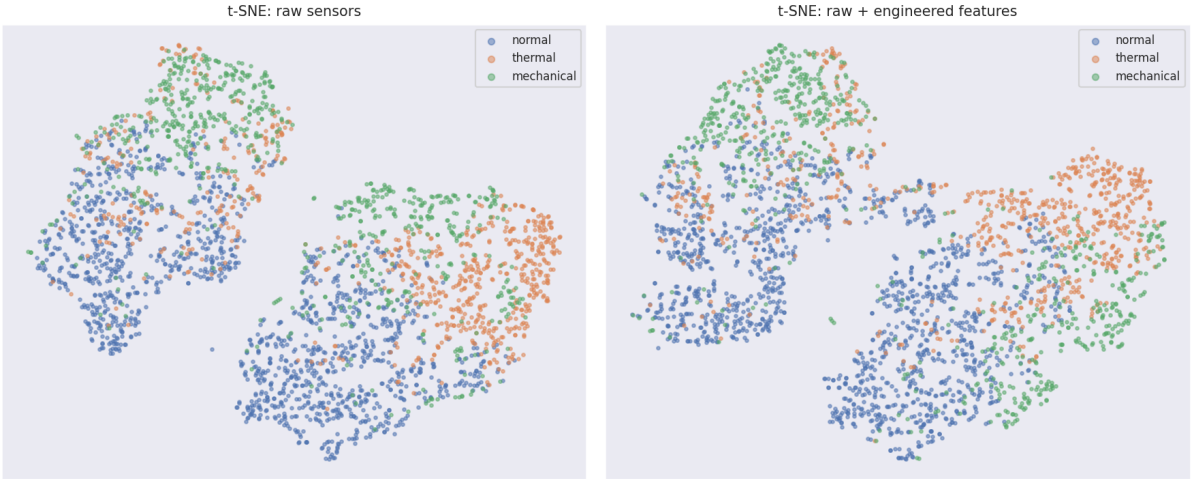


Figure 7: t-SNE before and after feature engineering.

12 Cross-validation of shortlisted models

Since a single 5-fold estimate can shift with the fold split, we re-evaluate the three finalists with 5 folds repeated 3 times: the quadratic logistic regression (0.8853 ± 0.0056), the forest (0.8777 ± 0.0051) and QDA (0.8476 ± 0.0068). The ordering stays the same. The differences between the top two models are comparable to those standard deviations, so the two are close to a tie, with the quadratic logistic still ahead.

13 Metrics comparison (full table)

We finally evaluate one representative configuration per family under accuracy, balanced accuracy and macro F_1 in a single cross-validation pass.

Model	Accuracy	Balanced acc.	Macro F_1
Logistic regression, quadratic basis (deg 2, $C = 1$)	0.8853	0.8681	0.8756
Random forest (depth 14)	0.8787	0.8554	0.8655
QDA (raw sensors, reg 0.1)	0.8476	0.8171	0.8299
Decision tree, cost-complexity pruned	0.8429	0.8194	0.8259
Decision tree, pre-pruned (depth 12, leaf 5)	0.8389	0.8126	0.8206
Logistic regression, linear ($C = 10$)	0.8293	0.8037	0.8104
LDA	0.8239	0.7893	0.8022
Gaussian naive Bayes (smoothing 0.1)	0.7535	0.7244	0.7273

Table 2: Machine condition: full metrics comparison.

From the table we see that the quadratic logistic regression leads on all three metrics, the forest follows closely, the post-pruned tree is the best tree, and Gaussian naive Bayes closes the list. Since the ranking is identical under all three metrics, the final choice does not depend on which metric is emphasised.

14 Submission creation and best model

For the submissions, we fit every model on all training rows and write one CSV per model, aligned to the competition's sample file by unit_id. The best model is also written as `submission.csv`. Every model tested has been uploaded (or is being uploaded) to the competition with a description of the model and its cross-validated metric.

Table 3: Best model and its submission.

Final model	CV score	Public LB	Submission file
Logistic regression, quadratic basis (deg 2, $C = 1$)	acc 0.8853	0.88812	`submission.csv`

The public scores that have returned follow the cross-validation ranking: quadratic logistic 0.88812, random forest 0.88125, QDA 0.85625, and the post-pruned tree 0.84937. The remaining model submissions (linear logistic, LDA, naive Bayes, pre-pruned tree and the forest-plus-QDA ensemble) are scheduled for the next upload day.

15 Conclusion

- The classes are split 2:1:1, but the per-class and balanced metrics show that accuracy is not hiding a weak class: the weakest recall is 83.3% on thermal failures, and the leading model is best on accuracy, balanced accuracy and macro F_1 at the same time.
- The sensors separate the classes in the way the failure physics suggests, and the engineered temperature difference and load ratios sharpen that separation.
- Quadratic boundaries matter: giving logistic regression a degree-2 basis takes it from 0.829 to 0.8853, ahead of QDA (0.848) and the forest (0.879). The forest is a close second.
- Tree tuning confirmed both pruning routes: pre-pruning peaks at depth 12 with leaf size 5 (0.839), and cost-complexity post-pruning with λ about 3×10^{-4} improves it to 0.8429 with 161 leaves. All tree variants stay behind the top two models.
- Reweighting, oversampling and SMOTE all reduced both accuracy and macro F_1, so the final models are unweighted.
- The best model is the quadratic-basis logistic regression (0.8853 ± 0.0056), and its submission scores 0.88812 on the public leaderboard.